

# CSE 4345: Big Data Analytics

Week 5, Part 1 — Handling Big Data Files: YARN, File Formats & Optimization

Department of Computer Science & Engineering  
Premier University, Chittagong

Semester 2026

# Course & Lecture Information

## Lecture Identity

**Course:** CSE 4345  
**Week / Part:** 5 of 11 | Part 1 of 2  
**CLO:** CLO3  
**PLO:** PLO(e), PLO(f)

## CLO3

*“Explain big data techniques, tools, and technologies used to store, manage, and analyse large datasets”*

## Course Progression

Week 3: HDFS (where data lives)  
Week 4: MapReduce / Pig / Hive / HBase (how

## Part 1 Covers

- Why storage format and scheduling matter at scale
- Hadoop 1.x vs. Hadoop 2.x: the YARN revolution
- YARN three-component architecture
- Container lifecycle and job execution flow
- File formats: CSV/Text, Avro, Parquet, ORC
- Row-oriented vs. columnar storage (why it matters)
- Compression codecs: Snappy, Gzip, LZO, Zstd
- Splittability: the critical property
- Format selection matrix for production
- Ivy League alignment

# Lecture Agenda

- 1 The Storage & Scheduling Problem
- 2 YARN: The Resource Management Revolution
- 3 File Formats: Row vs. Columnar Storage
- 4 Compression Codecs & Splittability
- 5 Format Selection in Production
- 6 Ivy League Alignment
- 7 Student Assessment Pool

## The Storage & Scheduling Problem

# Why Format and Scheduling Matter

## The Two Questions After HDFS & MapReduce

Once we can *store* data (HDFS) and *compute* over it (MapReduce), two engineering questions dominate production performance:

- 1 **Scheduling:** How do we share a 1,000-node cluster across 50 simultaneous jobs, different teams, and different frameworks (MapReduce, Spark, Tez) — fairly and efficiently?
- 2 **Format:** How do we store data so that a query reading 3 of 100 columns reads 3% of the data, not 100%?

## Real Cost at Scale

A 100-column table with 10 TB of data. A query selects 2

## Hadoop 1.x Bottleneck

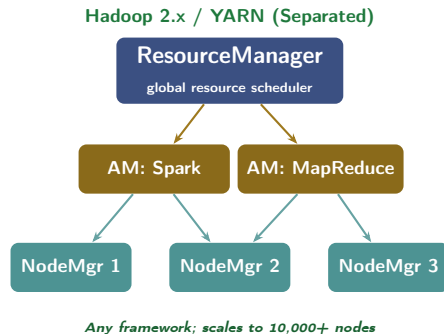
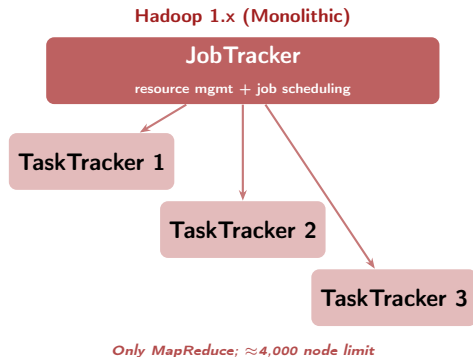
In Hadoop 1.x, MapReduce v1 bundled resource management with computation. Every cluster resource was managed by the **JobTracker**, which:

- Only ran MapReduce jobs (Spark, Tez, Storm had no path to cluster resources)
- Had a single JobTracker as a bottleneck ( $\approx 4,000$  node limit)
- Mixed cluster scheduling with job coordination

**The fix:** Separate resource management from computation. This became **YARN** in Hadoop 2.0 (2013).

## YARN: The Resource Management Revolution

# Hadoop 1 vs. Hadoop 2: The YARN Split



## The Key Architectural Decision

YARN introduced the **ApplicationMaster** — a per-job component that *negotiates* resources from the ResourceManager. This decoupling means any framework (Spark, Flink, Tez) can run on a YARN cluster without knowing how the cluster scheduler works.

# YARN Three-Component Architecture

## 1. ResourceManager (RM)

Global cluster resource scheduler. Runs on a dedicated master node.

- **Scheduler**: allocates *containers* to ApplicationMasters; pluggable (FIFO, Fair, Capacity)
- **ApplicationsManager**: accepts job submissions; starts the first container for each ApplicationMaster

## 2. NodeManager (NM)

Per-node agent running on every worker node.

- Launches and monitors **containers** on the local machine

## 3. ApplicationMaster (AM)

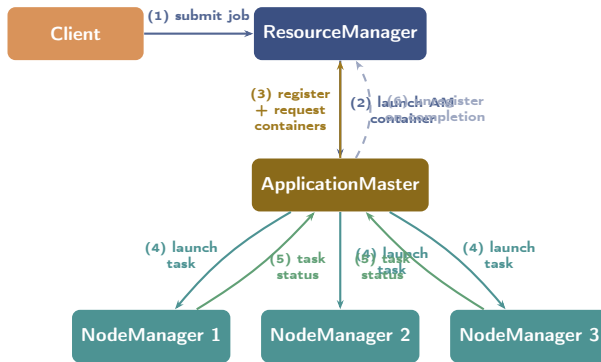
One per running job — the job's own scheduler.

- Requests resources (containers) from the RM on behalf of the job
- Tells the NM to launch specific tasks inside those containers
- Monitors task progress; re-requests containers for failed tasks
- Releases containers when tasks complete

**Example:** A Spark AM requests 100 containers (each 4 cores, 8 GB RAM) for Spark Executors; the RM grants 80 (cluster is busy); the AM assigns tasks to those 80 Executors.



# YARN Job Execution Flow



## YARN Fault Tolerance

If a **NodeManager fails**, the RM detects via missed heartbeats and marks all containers on that node as failed; the AM re-requests replacement containers. If the **AM itself fails**, the RM can restart a new AM (configured via `yarn.resourcemanager.am.max-attempts`).

# YARN Schedulers: Sharing the Cluster

## Three Built-in Scheduler Policies

- 1 **FIFO Scheduler**: jobs run in submission order; simple but not suitable for multi-tenant clusters (one long job starves all others)
- 2 **Capacity Scheduler** (default in HDP/CDH): cluster divided into queues with guaranteed capacity; e.g., Engineering queue = 60%, Finance queue = 40%; elastically borrows unused capacity
- 3 **Fair Scheduler** (default in Cloudera): all jobs get equal share of resources; latency-sensitive short jobs complete quickly even if a large job is running

## Bangladesh Context: a2i Cluster Sharing

The a2i (Access to Information) programme runs analytics on government data:

- **Queue A (60%)**: nightly DGHS health-record batch jobs (MapReduce)
- **Queue B (30%)**: Spark streaming for real-time agriculture data from DAE
- **Queue C (10%)**: ad-hoc Hive queries by data analysts

With **Capacity Scheduler**, queue B cannot monopolise resources even during peak streaming periods; queue A is guaranteed its 60%.

## File Formats: Row vs. Columnar Storage

# The Row vs. Columnar Storage Fundamental

## Row Store (CSV / JSON)

| id | name  | age | score |
|----|-------|-----|-------|
| 1  | Alice | 24  | 91    |
| 2  | Bob   | 31  | 78    |
| 3  | Carol | 27  | 88    |
| 4  | Dave  | 29  | 95    |

Query reads **ALL** columns

## Columnar Store (Parquet / ORC)

| id | name  | age | score |
|----|-------|-----|-------|
| 1  | Alice | 24  | 91    |
| 2  | Bob   | 31  | 78    |
| 3  | Carol | 27  | 88    |
| 4  | Dave  | 29  | 95    |

Query reads **ONLY** score column

## I/O Reduction at Scale

**Row store:** `SELECT score FROM users` reads ALL columns (id, name, age, score). **Columnar store:** reads ONLY the score column — physically skipping id, name, age on disk. For a 100-column table, this is a **97% I/O reduction** for a 3-column query.

# File Format Deep Dive: Avro, Parquet, ORC

| Format              |   | Orientation | Schema                 | Splittable | Compression    | Best For                       |
|---------------------|---|-------------|------------------------|------------|----------------|--------------------------------|
| <b>Text CSV</b>     | / | Row         | External               | With codec | None (default) | Portability, small data        |
| <b>JSON (lines)</b> |   | Row         | Self                   | No (raw)   | External       | APIs, log ingestion            |
| <b>Avro</b>         |   | Row         | Self-describing (JSON) | Yes        | Snappy/Deflate | Schema evolution, Kafka → HDFS |
| <b>Parquet</b>      |   | Columnar    | Self-describing        | Yes        | Snappy/Gzip    | Spark, Presto, ML pipelines    |
| <b>ORC</b>          |   | Columnar    | Self-describing        | Yes        | Zlib/Snappy    | Hive, highest compression      |

Why Avro for Schema Evolution?

Why Parquet for Analytics?

## Compression Codecs & Splittability

# Compression Codecs: The Trade-off Landscape

## Key Compression Codecs in Hadoop

| Codec  | Speed   | Ratio   | Splittable?                     |
|--------|---------|---------|---------------------------------|
| Snappy | Fast    | Medium  | No (but Parquet row groups are) |
| Gzip   | Slow    | High    | No                              |
| LZO    | Fast    | Medium  | Yes (with index)                |
| Bzip2  | Slowest | Highest | Yes                             |
| Zstd   | Fast    | High    | No (block-level)                |

## What Does “Splittable” Mean?

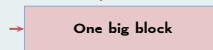
A file format is **splittable** if HDFS can start reading from any point in the file without needing to decompress from the beginning.

Parquet (splittable)



3 Mappers process 3 Row Groups in parallel

Gzip CSV (NOT splittable)



1 Mapper must read the entire file

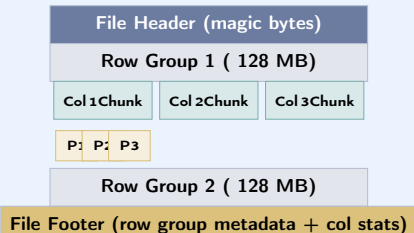
## Best Practice

Use **Parquet + Snappy** for analytics (Spark, Hive).

Use **Avro + Snappy** for streaming ingest (Kafka →

# Inside Parquet: Row Groups, Column Chunks, Pages

## Parquet Internal Layout



## Why This Layout Enables Predicate Pushdown

**Query:** WHERE score > 90

- 1 Read **File Footer** only: get column stats (min/max) per row group per column chunk
- 2 Row Group 1 has score in [60, 85] → **skip entire row group** (no I/O)
- 3 Row Group 2 has score in [85, 99] → must read this chunk
- 4 Within Row Group 2: read only the **score column chunk**

⇒ A query on 10 TB Parquet may physically read only **50 GB**.



## Format Selection in Production

# Format Selection Decision Matrix

| Scenario                        | Recommended Format | Compression | Reason                                                |
|---------------------------------|--------------------|-------------|-------------------------------------------------------|
| Streaming ingest (Kafka → HDFS) | Avro               | Snappy      | Schema evolution; row-append friendly                 |
| Spark analytics pipeline        | Parquet            | Snappy      | Columnar; predicate pushdown; Spark native            |
| Hive SQL warehouse              | ORC                | Zlib        | Best compression; Hive ACID support; vectorized reads |
| Data exchange / APIs            | Avro or JSON       | None        | Human-readable schema; broad ecosystem support        |

## Ivy League Alignment

# YARN & File Formats in Top University Curricula

## Harvard CS265 — Big Data Systems (Idreos)

Harvard frames file formats as a **fundamental algorithm design decision**:

- **I/O complexity model**: the cost of an algorithm is measured in the number of *I/O operations* (not time), and columnar formats directly reduce this measure
- **Access path selection**: just as a database optimizer chooses between a full table scan and an index, Parquet's predicate pushdown acts as a *lightweight index* embedded in the footer
- Harvard's benchmark: Parquet with predicate pushdown reduces I/O by 10–100 $\times$  for selective queries on wide tables (100+ columns)

**Harvard exam question**: “Prove that for a query selecting  $k$  columns from a  $c$ -column row-store table of  $n$  rows, the I/O complexity is  $\Theta(n \cdot c)$ , but  $\Theta(n \cdot k)$  for a column store.”

## MIT 6.830 — Database Systems

MIT compares YARN's DRF scheduler with classical database concurrency control:

- **DRF** (Ghodsi et al., 2011): generalises max-min fairness to multiple resource dimensions — the dominant resource of a job determines its fair share
- The YARN Capacity Scheduler is a practical implementation of hierarchical DRF with preemption

## CMU 15-721 — Advanced DB Systems

CMU covers Parquet/ORC as modern implementations of the **DSM (Decomposition)**

## Student Assessment Pool

## Instructions

Select the single best answer. Correct answers **bolded** for instructor reference.

**Q1.** Which component in YARN is responsible for **global resource allocation** across the entire cluster?

- ☒ (a) NodeManager   (b) ApplicationMaster   (c) **ResourceManager**   (d) NameNode

**Q2.** Which Hadoop file format stores data in **columnar** format and is best suited for analytical, read-heavy workloads?

- ☒ (a) Avro   (b) Text/CSV   (c) **Parquet**   (d) JSON

## Instructor Notes

**Q1:** Students confuse ApplicationMaster (per-job) with ResourceManager (global). Reinforce: **one RM per cluster**, one AM per running application. **Q2:** Students may choose ORC (also correct for Hive, but Parquet is the *broader* standard for analytics across Spark, Presto, and ML). Accept ORC as partially correct with justification.

**Q3.** What does “**splittable**” mean for a file format in Hadoop?

- ☐ a The file can be deleted in parts by HDFS
- ☒ b **Multiple Mappers can process different parts of the file simultaneously**
- ☐ c The file format supports splitting schema across multiple NameNodes
- ☐ d The file can be distributed across multiple data centres

**Q4.** YARN was introduced in which major Hadoop version, and what did it replace?

- ☐ a Hadoop 1.0; replaced the NameNode
- ☒ b **Hadoop 2.0; replaced the monolithic JobTracker with a decoupled ResourceManager + ApplicationMaster**
- ☐ c Hadoop 3.0; replaced DataNodes with Containers
- ☐ d Hadoop 2.0; replaced the Secondary NameNode with a Standby NameNode

### Instructor Notes

**Q3:** Option (b) is the correct answer. Splittability enables data-local task scheduling. **Q4:** Option (d) is a common distractor (HA NameNode is also a Hadoop 2.0 feature, but is unrelated to YARN). Students must know the JobTracker → RM + AM split.

## Instructions

State True or False and provide a **one-sentence justification** for full marks.

| Statement                                                                                                                          | Answer |
|------------------------------------------------------------------------------------------------------------------------------------|--------|
| 1. Parquet is a row-oriented file format optimised for write-heavy workloads.                                                      | False  |
| 2. YARN allows multiple computation frameworks such as Spark and Tez to share resources on the same Hadoop cluster.                | True   |
| 3. Gzip-compressed files are natively splittable in Hadoop, allowing multiple Mappers to process them in parallel.                 | False  |
| 4. ORC format is specifically optimised for Hive workloads and achieves among the highest compression ratios of any Hadoop format. | True   |



## Instructions

Answer each question in **100–150 words**. Include a diagram where indicated.

- D1.** Explain YARN's **three-component architecture** — ResourceManager, NodeManager, and ApplicationMaster. Draw a diagram showing how they interact when a Spark job is submitted to the cluster.
- D2.** What is **columnar storage**? Why does it improve performance for analytics queries that select only a subset of columns? Use a concrete example with numbers to illustrate the I/O reduction.
- D3.** Compare **Avro** and **Parquet**. Explain what “schema evolution” means and why Avro handles it better. In which scenarios would you use each?

## Instructions

Answer in **200–300 words** with step-by-step reasoning. Marks awarded for correct process, not just final answer.

### A1. Data Lake Performance Migration:

A Dhaka-based telecom company stores 10 TB of daily call-detail records (CDR) on HDFS as gzip-compressed CSV files. Their data engineers report that a Hive query selecting 4 of 80 columns takes 6 hours.

- Identify **two specific reasons** why this configuration is slow
- Propose a migration strategy: new format, compression codec, and partitioning scheme
- Estimate the expected I/O reduction (show your calculation)
- What YARN scheduler would you use and why, given that multiple teams share the cluster?

### A2. YARN Failure Trace:

A NodeManager (NM2) hosting 5 running containers for a MapReduce job fails mid-execution. Trace the complete failure recovery sequence: which components detect the failure, in what order, what state changes occur in the ResourceManager, and how are the 5 containers' tasks restarted.

# Textbook & Reading References

## Primary Textbooks for Week 5

- **[TW]** Tom White. *Hadoop: The Definitive Guide*, O'Reilly, 4th ed., 2015. **Ch. 3, 4, 5**
  - Ch. 3: HDFS · Ch. 4: YARN · Ch. 5: Hadoop I/O (compression, serialization, file formats)
- **[SA]** Acharya & Chellappan. *Big Data and Analytics*, Wiley, 2015. **Ch. 6**
  - Ch. 6: YARN and Resource Management

## Supplementary (Covered in Week 5, Part 2 — Seminal Paper & Case Study)

- Vavilapalli, V.K. et al. (2013). **Apache Hadoop YARN: Yet Another Resource Negotiator**. *ACM SoCC 2013*. **[PRIMARY SEMINAL PAPER]**
- Ghodsi, A. et al. (2011). **Dominant Resource Fairness: Fair Allocation of Multiple Resource Types**. *USENIX NSDI 2011*. **[DRF algorithm underlying YARN Fair Scheduler]**
- Apache Parquet Documentation: [parquet.apache.org](https://parquet.apache.org) — Row groups, column chunks, predicate pushdown
- Apache Avro Documentation: [avro.apache.org](https://avro.apache.org) — Schema evolution, reader/writer schema

# Questions & Discussion

Week 5, Part 1 | CSE 4345 Big Data Analytics

---

*“Storing data in HDFS is cheap. Storing it in the wrong format is expensive — you pay the cost every single time you query it.”*

— Julien Le Dem, Apache Parquet co-creator (Twitter, 2014)

## Next Lecture: Week 5, Part 2

Seminal Paper: Vavilapalli et al. (2013) — *“Apache Hadoop YARN: Yet Another Resource Negotiator”*

Design decisions · Capacity Scheduler internals · DRF algorithm

Case Study: LinkedIn’s schema evolution challenge and Avro adoption (5 TB/day scale)

**Reading before Part 2: [TW] Ch. 4 · YARN paper (see references)**